SRS REPORT
for
Campus Event Discovery App

Team Name: 1
Team Member 1: Ojashwi Shrestha
Team Member 2: Adamson Buliboli
Team Member 3: Navroop Kaur
Team Member 4: Anesu David Pasipanodya
Team Member 5: Mahak

1. Introduction (OJU)

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed description of the Campus Event Discovery App. This document outlines the system's functional and non-functional requirements, system design, and overall architecture. It serves as a reference for developers, stakeholders, and team members throughout the development lifecycle.

1.2 Scope

The Campus Event Discovery App is a web-based application designed to centralize and simplify access to campus events. The system allows students to browse, search, and save events, while organizers can create and manage event listings. The platform aims to improve event visibility and student engagement by providing a user-friendly and accessible interface.

1.3 Project Objectives

The main objectives of this project are:

- To develop a centralized platform for discovering campus events
- To enable users to browse, search, and filter events
- To allow users to save and track events of interest
- To provide organizers with the ability to create and manage events
- To design an intuitive and user-friendly interface

1.4 Problem Description

In many educational institutions, event information is distributed across multiple platforms such as emails, social media, and physical posters. This fragmentation makes it difficult for students to stay informed about upcoming activities. As a result, many events experience low participation due to lack of awareness. There is a need for a centralized system that consolidates event information and improves accessibility.

1.5 Methodology

The project will follow the Software Development Lifecycle (SDLC) using an agile approach. The development process will include discovery, requirement analysis, system design, development, testing, and deployment. Iterative development will be used to continuously improve the system based on feedback and testing results.

2. Business and System Overview (Mahak)

2.1 Business Description

The Campus Event Discovery App is an EdTech web-based application aiming to enhance student engagement through an efficient system for discovering events. Campuses often host academic workshops, social events, career sessions, and club activities, but students often don't participate in these events due to poor visibility and information dissemination.

Now, students receive information about events through various channels, such as emails, posters, social media, and university internal portals. However, this has created confusion and decreased accessibility for students.

The purpose of this system is to:

- Enhance the relationship between event organizers and students
- Increase student engagement in events hosted within the university
- Provide an efficient system for event discovery for students

This system is intended to act as a bridge between event organizers and students, allowing them to access information easily.

2.2 Current System (Existing Process)

The current system for event discovery is through a manual system. The information is disseminated through various channels, such as:

- Sending emails to students
- Social media platforms like Facebook and Instagram
- Physical posters and notice boards
- University internal portals

Limitations of the Current System:

- ❌ No centralized system for event discovery

- ❌ Information is disseminated through various channels, creating confusion
- ❌ Students are missing out on various events
- ❌ No options for filtering information
- ❌ No options for personalization
- ❌ No options for organizers to track students' engagement

Due to these disadvantages, students often face information overload, and some students are unaware of events, resulting in poor student engagement.

2.3 Proposed System

The proposed system is a full-stack web application for campus event management.
The following are the features of the proposed system:

- Event creation and management by event organizers
- Event browsing and filtering for the student community
- Search facility
- Bookmarking/saving events
- Personalized dashboard

The system will enable the following features for the students and event organizers:

- Students will be able to easily find events based on their interests.
- The event organizers will be able to easily publish the events.

This system will directly address the limitations of the existing system by providing a single system.

2.4 System Overview

The Campus Event Discovery App will be based on a three-tier architecture system.

1. Presentation Layer (Frontend)
   This layer will be developed using React.js.

2. Application Layer (Backend)
   This layer will be developed using Node.js/Express.

3. Data Layer (Database)
   This layer will be developed using MySQL or MongoDB.

The system will have the following functions:

- Authentication (login and register)
- List and filter events
- Search and bookmark events
- Personalized dashboard

The system will be hosted on a cloud platform.

3. Requirements Analysis

3.1 Business Requirements (mahak)

The business requirements describe the high-level goals and objectives to be achieved by the system.

1. Centralized Event Management

The system will provide a single platform where all the campus events will be accessible in one place.

2. Improved Student Engagement

The system will increase the involvement and participation of the students by providing them easy access to the events.

3. Efficient Communication

The system will provide effective communication between the event organizers and the students.

4. Personalization

The system will allow the user to personalize the events by filtering them based on the user's interests and saving the events.

5. Real-Time Updates

The system will provide the latest information about the events, such as updates or cancellations.

6. Ease of Use

The system will provide a user-friendly experience by being easy to use and accessible on different devices.

7. Scalability

The system should be able to accommodate:

- A large number of users
- Multiple events occurring at the same time

8. Data Security and Privacy

The system must provide:

- Secure user authentication
- Protection of user data

3.2 User Requirements (Oju)

Students should be able to browse events, view event details, search and filter events, and save events for future reference. If user accounts are implemented, students should be able to register, log in, and manage their saved events.

Event organizers should be able to create, update, and delete event listings by providing relevant event information such as title, date, time, location, and category.

The detailed interaction flow of users with the system is illustrated in the use case diagram provided in Section 3.2.1.

3.2.1 Use Case Diagram (David)

3.2.2 Use Case Descriptions (David)

Use Case Overview

The table below summarises all ten use cases across both actor types.

ID	Use Case	Actor	Description
UC-01	Register / Log In	Student & Organiser	Create an account or authenticate into the system
UC-02	Browse Events	Student	Scroll and view the full upcoming events feed
UC-03	Search Events	Student	Find events using keywords across title, description, category
UC-04	Filter Events	Student	Narrow the event list by category, date, or location
UC-05	View Event Details	Student	See full information for a selected event
UC-06	Save / Bookmark Event	Student	Save an event to a personal list for later reference

UC-07	View Organiser Dashboard	Organiser	View event listings, engagement metrics, and quick actions
UC-08	Create Event	Organiser	Publish a new event listing to the platform
UC-09	Edit Event	Organiser	Update the details of an existing event
UC-10	Delete Event	Organiser	Permanently remove an event from the system

Note: UC-01 is shared by both actors. UC-02 to UC-06 are student-facing. UC-07 to UC-10 are organiser-facing.

Student Use Cases

UC-01 — Register / Log In | Actor: Student & Organiser	
Description	A new user creates an account or an existing user authenticates to access personalised features.
Trigger	User opens the app and selects Register or Log In.

Steps	1. User selects Register or Log In. 2. Registration: enters name, university email, password, and role (Student or Organiser). 3. System validates input and checks email is not already registered. 4. System creates account and redirects user to their dashboard. 5. Login: user enters email and password; system authenticates and creates a session.
Alternate Flows	**Email already registered:** Prompts user to log in instead. **Invalid credentials:** Error shown; account locked after 5 failed attempts. **Missing fields:** Fields highlighted; submission blocked. **Forgot password:** Reset email triggered via 'Forgot password' link.
Postcondition	User is authenticated and redirected to their role-appropriate dashboard.
NFRs	Passwords stored with bcrypt hashing — never plain text. Session tokens expire after 24 hours of inactivity. Forms must respond in under 2 seconds. Email must match a valid university domain format.

UC-02 — Browse Events | Actor: Student	
Description	A student views the full list of upcoming campus events in chronological order. Login is not required.

Trigger	Student opens the app or navigates to the Events page.
Steps	1. Student navigates to the Events page. 2. System retrieves all upcoming events sorted by earliest date. 3. Event cards display: title, date, time, location, and category. 4. Student scrolls through the feed. 5. Student clicks an event card to view full details (UC-05).
Alternate Flows	**No events:** Empty state message shown; organisers prompted to create events. **Load failure:** Error shown with a retry button. **Not logged in:** Browsing available; save/bookmark options greyed out with login prompt.
Postcondition	Student can see all upcoming events and select any for further detail.
NFRs	Feed must load within 3 seconds on a standard connection. Mobile-responsive from 320px screen width. Past events automatically hidden from the feed.

UC-03 — Search Events | Actor: Student	
Description	A student uses a keyword search to find relevant events without manually browsing.

Trigger	Student types a keyword into the search bar.
Steps	1. Student navigates to the search bar. 2. Student types a keyword (e.g. 'workshop', 'careers'). 3. System queries title, description, and category fields. 4. Matching events returned sorted by date. 5. Student selects an event to view full details.
Alternate Flows	**No results:** Message shown with suggestion to browse all events. **Empty search:** Defaults to full browse view. **Under 3 characters:** Search not executed until minimum threshold met. **Partial match:** Closest matches returned; matched terms highlighted.
Postcondition	Matching events displayed, filtered by the entered keyword.
NFRs	Results returned within 3 seconds. Case-insensitive search. Minimum 3 characters required before search executes. Matched terms visually highlighted in results.

UC-04 — Filter Events | Actor: Student	
Description	A student applies one or more filters to narrow the event list by category, date range, or location.

Trigger	Student opens the filter panel and selects filter criteria.
Steps	1. Student opens the filter panel on the Events page. 2. Student selects filters: Category, Date range, and/or Location. 3. System applies filters to the database query. 4. Only matching events are displayed. 5. Student can adjust or clear filters at any time.
Alternate Flows	**No matches:** 'No events match your filters' shown with a Clear Filters button. **All filters cleared:** Returns to full browse view. **Overly restrictive:** Live count of matching events shown as filters are applied.
Postcondition	Student sees a refined list matching their selected filter criteria.
NFRs	Filter operations respond in under 2 seconds. Multiple filters combinable simultaneously. Active filters clearly displayed with individual remove options.

UC-05 — View Event Details | Actor: Student	
Description	A student views the complete information for a specific event selected from browse, search, or filter results.

Trigger	Student clicks on an event card.
Steps	1. Student clicks an event card. 2. System retrieves the full event record. 3. Detail page shows: title, description, date, time, location, category, organiser name, and notes. 4. Student reads details and decides whether to attend. 5. Student optionally bookmarks the event (UC-06) or navigates back.
Alternate Flows	**Event not found:** 'Event not found' message shown; redirected to browse. **Event has passed:** Event displayed but marked as 'Past event'. **Missing organiser info:** 'Organiser information not available' shown.
Postcondition	Student has all information needed to decide whether to attend.
NFRs	Detail page loads within 2 seconds. Accessible and readable on mobile devices. All event fields validated at creation to avoid blank pages.

UC-06 — Save / Bookmark Event | Actor: Student	
Description	A logged-in student saves an event to their personal bookmarks list for easy access later.

Trigger	Student clicks the bookmark or save icon on an event.
Steps	1. Student clicks the bookmark icon on a card or detail page. 2. System checks the student is authenticated. 3. Event saved to the student's personal bookmarks in the database. 4. Bookmark icon updates to indicate saved; confirmation shown. 5. Student can access saved events via their dashboard.
Alternate Flows	**Not logged in:** 'Log in to save events' prompt shown with login link. **Already bookmarked:** Icon shown as active; option to remove bookmark offered. **Remove bookmark:** Bookmark deleted; icon updated; event removed from saved list.
Postcondition	Event saved to student's bookmarks and accessible from their dashboard.
NFRs	Bookmark action completes within 1 second. Bookmark state persists across sessions (stored in database). Bookmarkable from both card view and detail page.

Organiser Use Cases

UC-07 — View Organiser Dashboard | Actor: Organiser	
Description	A logged-in organiser views their events, engagement metrics, and quick-access actions.
Trigger	Organiser navigates to the Dashboard page after logging in.
Steps	1. Organiser logs in and navigates to the Dashboard. 2. System retrieves all events created by this organiser. 3. Upcoming events shown with title, date, time, location, RSVP count, and view count. 4. Past events shown in a separate section with final metrics. 5. Quick-action buttons (Edit, Delete) displayed per event. 6. Summary stats shown: total events, total RSVPs, most popular event. 7. Organiser selects an action (e.g. Edit or Create new event).
Alternate Flows	**No events yet:** 'No events created yet' shown with prominent Create Event button. **All events past:** Past events section shown; prompted to create new event. **Metrics unavailable:** Listing shown without metrics; 'Analytics not yet available' note. **Session expires:** Redirected to login; dashboard state restored on re-auth.

Postcondition	Organiser has a full view of their events and metrics with direct access to management actions.
NFRs	Dashboard loads within 3 seconds. Metrics update in real time or within 60 seconds of a student action. Only shows events created by the authenticated organiser. Quick-action buttons accessible without leaving the dashboard. Fully functional on mobile devices.

UC-08 — Create Event | Actor: Organiser	
Description	An organiser creates and publishes a new event listing so students can discover it.
Trigger	Organiser clicks 'Create Event' from their dashboard or navigation.
Steps	1. Organiser clicks 'Create Event'. 2. System displays the event creation form. 3. Organiser fills in: title, description, date, start/end time, location, and category. 4. Organiser optionally adds notes. 5. Organiser clicks 'Publish Event'. 6. System validates all required fields and confirms date is in the future.

	7. Event saved and immediately visible in the student feed. 8. Organiser redirected to confirmation screen with a link to the published event.
Alternate Flows	**Required fields missing:** Missing fields highlighted; submission blocked. **Past date entered:** 'Event date must be in the future' validation error shown. **Saved as draft:** Event saved but not published to student feed. **Session expires mid-form:** Re-login prompted; form data preserved where possible.
Postcondition	New event published and immediately visible in the student feed.
NFRs	Form completable on mobile devices. Published events appear in student feed within 5 seconds. Only users with Organiser role can access the creation form. All text fields enforce a maximum character limit.

UC-09 — Edit Event | Actor: Organiser	
Description	An organiser updates details of an existing event they own.

Trigger	Organiser selects 'Edit' on one of their existing events.
Steps	1. Organiser navigates to 'My Events' in the dashboard. 2. System displays all events created by the organiser. 3. Organiser clicks 'Edit' on the target event. 4. System displays the edit form pre-populated with current event details. 5. Organiser modifies relevant fields. 6. Organiser clicks 'Save Changes'. 7. System validates and saves changes. 8. Student feed updated immediately; confirmation shown.
Alternate Flows	**Editing another organiser's event:** Access denied: 'You do not have permission to edit this event'. **Required field cleared:** Save blocked; empty field highlighted. **Cancel without saving:** Changes discarded; original event unchanged.
Postcondition	Event updated with new information, reflected immediately in the student feed.
NFRs	Edit form pre-populated with current event data. Changes reflected in student feed within 5 seconds. Edit restricted to the creating organiser or a system administrator.

UC-10 — Delete Event | Actor: Organiser

Description	An organiser permanently removes an event listing they own from the platform.
Trigger	Organiser selects 'Delete' on one of their existing events.
Steps	1. Organiser navigates to 'My Events' in the dashboard. 2. System displays the organiser's event list. 3. Organiser clicks 'Delete' on the target event. 4. System shows confirmation dialog: 'Are you sure? This cannot be undone.' 5. Organiser confirms by clicking 'Yes, delete'. 6. System removes the event from the database. 7. Event removed from student feed immediately; success confirmation shown.
Alternate Flows	**Cancel at confirmation:** Dialog dismissed; event remains unchanged. **Deleting another organiser's event:** Access denied: 'You do not have permission to delete this event'. **Event has active bookmarks:** Event deleted and removed from all students' saved lists; students not notified (v1).
Postcondition	Event permanently removed from the database and student feed.
NFRs	Deletion requires an explicit confirmation step — no single-click delete. Deleted events automatically removed from all student bookmark lists. Deletion restricted to the creating organiser or a system administrator.

	Consider soft-delete (archive) in future versions for recovery.

3.3 System Requirements

3.3.1 Functional Requirements

3.3.2 Non-Functional Requirements

---

4. System Configuration

This section outlines the hardware, software, and development environment requirements for building and running the Campus Event Discovery App. These specifications ensure that all team members work in a consistent environment and that the deployed application meets performance expectations.

4.1 Hardware Requirements

The following hardware specifications apply to the machines used by developers during the development phase, as well as the server infrastructure needed for deployment.

4.1.1 Development Machines

Component	Minimum Specification
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or equivalent — 2.0 GHz or higher

RAM	8 GB minimum (16 GB recommended for running frontend, backend, and database simultaneously)
Storage	256 GB SSD (sufficient space for Node.js, npm packages, database, and project files)
Network	Stable broadband internet connection (required for package installation and cloud deployment)
Display	1280 x 720 resolution minimum (for effective UI/UX design work)
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04 LTS

4.1.2 Server / Hosting Infrastructure (Cloud)

(to be decided)

4.2 Software Requirements

The application relies on the following software components across the presentation, application, and data layers as defined in the three-tier architecture (Section 2.4).

4.2.1 Frontend

Technology	Details
React.js	JavaScript library for building the user interface (component-based architecture)
HTML5 / CSS3	Markup and styling for responsive, accessible web pages
Axios / Fetch API	HTTP client for making API calls from the frontend to the backend

React Router	Client-side routing for navigating between pages (Events, Dashboard, Login, etc.)
Node Package Manager (npm)	Dependency management for frontend packages

4.2.2 Backend

Technology	Details
Node.js (v18+)	Server-side JavaScript runtime environment
Express.js	Lightweight web framework for building RESTful APIs
bcrypt	Password hashing library for secure authentication
JSON Web Tokens (JWT)	Token-based authentication for managing user sessions
dotenv	Environment variable management for sensitive configuration
CORS	Cross-origin resource sharing middleware for frontend-backend communication

4.2.3 Database

Technology	Details
MySQL	Relational (MySQL) database for storing events and user data
Sequelize (if MySQL)	ORM for defining and querying database models with JavaScript

4.2.4 Authentication & Security

Technology	Details
JWT (JSON Web Tokens)	Stateless session management; tokens expire after 24 hours of inactivity
bcrypt	Industry-standard password hashing; passwords are never stored as plain text
HTTPS	All production traffic encrypted via SSL/TLS

4.2.5 Cloud Deployment

(to be decided)

4.3 Development Environment

All team members will configure the same development environment to ensure consistency across machines and avoid environment-specific bugs.

4.3.1 Code Editor and IDE

Tool	Purpose
Visual Studio Code (VS Code)	Primary code editor (free, cross-platform, strong JavaScript/React support)
Prettier	Automatic code formatter for consistent code style across the team

4.3.2 Version Control

Tool	Purpose
Git	Distributed version control system for tracking code changes

GitHub	Remote repository hosting, pull requests, code reviews, and issue tracking

4.3.3 API Testing and Development Tools

Tool	Purpose
Postman	Testing and documenting REST API endpoints during backend development
Thunder Client (VS Code) (Backup)	Lightweight alternative to Postman, available as a VS Code extension

4.3.4 Project Management and Collaboration

Tool	**Purpose**
Jira	Agile task management using Kanban boards for sprint planning
WhatsApp	Team communication, daily stand-ups, and file sharing
Figma	UI/UX wireframing and prototyping for interface design (Section 7)
Google Docs	Collaborative documentation and meeting notes

4.3.5 Local Development Setup Summary

Each developer will perform the following steps to set up their local environment:

- Install Node.js (v18 LTS) and npm from nodejs.org

- Clone the team repository from GitHub
- Run npm install in both the /client (React) and /server (Express) directories
- Configure a .env file with local database credentials and JWT secret key
- Start the backend server using npm run dev (nodemon for auto-reload)
- Start the React frontend using npm start
- Verify the application runs locally on localhost:3000 (frontend) and localhost:5000 (backend API)

This setup ensures every team member can develop and test all features locally before committing changes, reducing integration conflicts and improving overall code quality.

---

5. System Design

5.1 System Architecture

The Campus Event Discovery App is built on a three-tier client-server architecture, separating concerns across the Presentation Layer, Application Layer, and Data Layer. This approach promotes modularity, maintainability, and scalability throughout the system.

5.2 Data Flow Diagram (DFD) (david)

5.3 Database Design (adam)

5.3.1 Entity Relationship Diagram (ERD) (david)

5.3.2 Tables / Data Model (adam)

5.3.3 Data Dictionary (adam)

5.4 Class Diagram (david)

---

6. Security and Data Protection

6.1 Data Encryption

The system will ensure that user data is protected during transmission and storage.

- All communication between the frontend and backend will use HTTP to secure data in transit.
- User passwords will be stored using bcrypt hashing and will never be stored in plain text.
- Authentication will be handled using JSON Web Tokens (JWT), which will expire after a defined period to improve security.
- Sensitive configuration data (such as secret keys, database connection, etc) will be stored using environment variables.

6.2 Data Anonymization

The system will protect user privacy by limiting exposure of personal data.

- User email and sensitive information will not be publicly visible.
- Only necessary information such as organiser name will be shown in event details.
- User activity such as saved/bookmarked events will remain private to each user.

6.3 Data Protection Impact Assessment (DPIA)

The system identifies the following potential risks and also has the list of basic security measures to reduce them.

Risks:

- Unauthorized access to user accounts
- Unauthorized editing or deletion of events
- Misuse of user data

Mitigation:

- Secure login using JWT and encrypted passwords
- Role-based access:
    - Students can browse and save events
    - Organisers can create, edit, and delete events
- Only the organiser who created an event can modify or delete it
- Protected API routes to ensure only authenticated users can perform actions

---

7. User Interface Design

7.1 Design Principles

7.2 Storyboards / Wireframes

7.3 Input Forms

7.4 Output Screens / Reports

---

8. Work Breakdown Structure and Project Planning

8.1 Work Breakdown Structure (WBS)

8.2 Milestones

8.3 Project Schedule (Gantt Chart)

8.4 Team Roles and Responsibilities

---

9. Implementation Plan

9.1 Development Approach

9.2 Deployment Plan

---

10. Test Plan

10.1 Testing Strategy

10.2 Test Cases

10.3 Expected Results

---

11. Evaluation and Improvements

11.1 Suggested Improvements

11.2 Response to Lecturer Feedback

---

12. Summary of Deliverables

---

13. Conclusion

---

14. References

---

15. Appendices

- Use Case Diagram
- DFD
- ERD
- Screenshots / UI Designs
- Additional Documents
-